AI-Powered University Student Services Assistant

Master Project Guide: Detailed Tech Stack, File-by-File Breakdown & Interview Preparation

Project Focus	Hybrid RAG + Deterministic Rule Engine for Academic & Student Services
Tech Stack	FastAPI, LangGraph, ChromaDB, SQLite3, React 18 + Vite, Sentence Transformers, Ollama / Mock LLM
Architecture Pattern	Decoupled State Machine (LangGraph) + Pure Python Source Precedence + Dynamic DB Tools
Document Purpose	Exhaustive Code & Component Walkthrough for Technical Interview Preparation

EXECUTIVE ARCHITECTURE OVERVIEW

This project provides a production-grade blueprint for building zero-hallucination AI assistants in highly regulated domains. Every single academic calculation (attendance %, eligibility rules, exam backlogs) is performed by Python tools against SQLite, while policy selection uses a 5-step precedence algorithm.

1. Complete Tech Stack Breakdown & Justifications

Understanding why each technology was selected over alternatives is essential for senior technical interviews. Below is the comprehensive breakdown:

Technology	Category / Layer	Why Selected & Key Advantage over Alternatives
FastAPI	Backend Framework	Asynchronous performance, native Pydantic type safety, automatic OpenAPI documentation (/docs), lightweight dependency injection.
LangGraph	Agent Orchestration	Cyclic state machine control (StateGraph). Unlike standard linear LangChain chains, LangGraph allows explicit state routing, branch conditions, and step isolation.
ChromaDB	Vector Database	Persistent local vector database with embedding function abstraction. Perfect for embedded execution without requiring external cloud vector services.
SQLite 3	Relational DB	Embedded zero-config relational store with foreign key enforcement and CHECK constraints. Fast local persistence for student profiles, marks, and audit logs.
Sentence Transformers	Embedding Model	Uses all-MiniLM-L6-v2 for 384-dimensional dense semantic vectors. Custom term-frequency SHA-256 fallback included for zero-dependency offline mock mode.
React 18 + Vite	Frontend Framework	Modern component-based UI with instant hot module replacement (HMR), TypeScript integration, and clean state handling.
Pydantic v2	Data Validation	Strict schema enforcement for request/response bodies, intent routes, and audit records with high-speed Rust-backed validation.
Ollama / Llama 3.1	LLM / Explainer Layer	Local open-weight LLM execution. Constrained purely to explaining computed evidence without deciding policies or calculating math.
Pytest & Vitest	Testing Frameworks	Backend unit and integration test suite (test_core.py) and frontend test runner for continuous quality assurance.
ReportLab	PDF Generation	Programmatic PDF generation engine for rendering complete project technical guides and interview documentation.

2. Exhaustive File-by-File Breakdown

Here is the complete analysis of every file in the codebase, detailing its responsibility, key functions, and interaction with other components:

■ backend/app/main.py — *FastAPI Application Gateway & REST Endpoints*

Entry point for the backend API server. Defines routes: POST /ask (query handler with X-Student-Id header authentication), POST /ingest (document upload & parsing), GET /health (subsystem health check), GET /audit/{trace_id} (audit trail retrieval), POST /reindex (vector store rebuild), GET /sources, GET /rules, DELETE /sources/{doc_id}. Uses require_student() to enforce identity boundaries.

■ backend/app/graph.py — *LangGraph State Machine Orchestration*

Defines the workflow graph using StateGraph(State). Contains three main nodes: 1) router (normalizes query, extracts course regex e.g. CS201, classifies intent into 8 categories), 2) retrieve_node (executes Chroma vector search, section matching, content deduplication, and invokes source_precedence.resolve), 3) answer (executes tools for personal data/eligibility, extracts exact evidence sentences for policy queries, constructs hypothetical in-memory states for multi-step queries, or returns refusal/not_found).

■ backend/app/policy/source_precedence.py — *Deterministic Source Precedence Engine*

Contains resolve() and _applies(). Implements a pure Python 5-step policy ordering algorithm: 1) Applicability filter (effective_from <= as_of_date < effective_to, programme, batch), 2) Explicit supersession filter (removes documents superseded by newer ones), 3) Authority level hierarchy rank (Level 1 Regulations > Level 2 Circulars > Level 3 Notices), 4) Recency sorting by effective date, 5) Conflict detection (flags equal-precedence documents for safe review).

■ backend/app/tools.py — *Deterministic Database Tools & Calculations*

Standalone Python functions querying SQLite: profile(student_id) (fetches demography), attendance(student_id, course_code) (queries held/attended classes and computes percentage = round(attended/held*100, 2)), result(student_id, course_code), rules(as_of_date), _rule() (locates valid threshold), and eligibility(student_id, course_code, as_of_date, hypothetical_pass) (evaluates attendance threshold and result state). Supports hypothetical passes without DB mutation.

■ backend/app/rag.py — *Persistent Vector Store & Section Parser*

Manages persistent ChromaDB collection university_documents_v2. Contains Embeddings class with Sentence Transformers or SHA-256 term-frequency hashing fallback. Implements sections(text) regex parser recognizing generic headers (e.g. EXPERIMENT 1, SECTION 7, MODULE 3), chunks(text, size=800, overlap=120), index_document(), remove_document(), rebuild(), and search().

■ backend/app/retrieval.py — *Hybrid Keyword Retrieval & Section Extractor*

Provides retrieve(question) with fallback keyword scoring across document texts when vector hits are sparse. Implements exact_section(question) to extract full heading-delimited section blocks for exact matches (e.g. Experiment 1), and citation(d) to format citation metadata.

■ backend/app/db.py — *SQLite Database Schema & Connection Pool*

Defines SQL DDL schema with PRAGMA foreign_keys=ON. Creates tables: students, courses, attendance, results, rule_registry, source_register, documents, and audits. Enforces CHECK constraints (cgpa BETWEEN 0 AND 10, classes_attended <= classes_held, total_marks = internal + external). Provides conn() context manager for thread-safe transactions.

■ backend/app/schemas.py — *Pydantic Models & Data Transfer Objects*

Defines data contracts: AskRequest (question, optional as_of_date), Citation, ToolResult, AskResponse (trace_id, answer, answer_type, citations, tool_results, conflicts, latency_ms, applied_rules), Route (intent, entities), and IngestResponse.

■ backend/app/seed.py — *Synthetic Database & Policy Initialization*

Seeds SQLite with 30 synthetic students (S1001-S1030), 6 courses (CS201, CS202, EC201, EC202, HS201, MA201), attendance rows, exam results, synthetic policy documents (ACAD-REG-2023, ACAD-REG-2024, EXAM-RULES-2024, ATT-CIRCULAR-2024, CSE-NOTICE-2024), rule registry entries, and populates Chroma vector store.

■ backend/app/llm.py — *Optional Explanation Layer Adapter*

Contains explain(answer, evidence, tool_results). In MOCK_LLM=true mode, returns verified answer directly with zero external API calls. When MOCK_LLM=false, calls local Ollama endpoint (llama3.1:8b) with strict prompt constraints prohibiting document instruction execution or prompt leakage.

■ backend/app/config.py — *Environment & Central Settings*

Loads environment variables from .env using python-dotenv. Defines root paths (DATA_DIR, DB_PATH, CHROMA_PATH, DOCS), Ollama parameters (OLLAMA_BASE_URL, OLLAMA_MODEL), retrieval settings (TOP_K, MAX_UPLOAD_MB), embedding model selection, and MOCK_LLM toggle.

■ backend/tests/test_core.py — *Backend Test Suite (Pytest)*

Comprehensive unit and integration tests: test_attendance_is_deterministic, test_supersession_wins, test_equal_sources_conflict, test_ask_and_audit, test_other_student_refused, test_as_of_date_selects_historical_version, test_future_policy_is_not_applicable, test_what_if_does_not_change_database, test_new_document_is_retrievable, test_section_parser_does_not_bleed.

■ frontend/src/main.tsx — *React Dashboard Component*

Main frontend application. Provides demo student selector dropdown (S1001-S1003), question input textarea, optional as_of_date picker, quick-try example buttons, HTTP POST fetch call to /ask with X-Student-Id header, and renders response badges, citations, applied rules, and formatted tool execution JSON.

■ frontend/src/style.css — *Application Styling & Badge Themes*

CSS styling featuring modern flexbox layouts, color variables, typography, response type badge colors (green for retrieved/calculated, amber for not_found/refused, red for conflict_flagged), and dark code block styles.

■ docker-compose.yml — *Multi-Container Docker Orchestration*

Defines services for backend (FastAPI on port 8000 with mounted ./data volume) and frontend (Nginx web server on port 5173 dependent on backend).

■ scripts/generate_project_guide.py — *PDF Documentation Generator*

ReportLab script that programmatically compiles this comprehensive project guide and interview preparation document into university_student_services_project_guide.pdf.

■ evaluation/evaluate.py — *System Quality & Benchmark Evaluator*

Evaluates system performance against evaluation/dataset.json (20 test cases). Calculates correctness %, citation accuracy %, tool correctness %, abstention accuracy %, top-k=3/5 retrieval hit rates, and P50/P95 latency.

3. Step-by-Step Request Lifecycle & Execution Walkthrough

To demonstrate deep technical understanding during an interview, trace a sample query from click to screen:

Sample User Action: Student S1001 selects CS201 attendance question and clicks "Ask Assistant"

• **Step 1: Frontend Request Dispatch (React)**: main.tsx captures student ID ("S1001") and question text ("What is my attendance in CS201?"). Sends HTTP POST to http://localhost:8000/ask with JSON body {"question": "..."} and request header X-Student-Id: S1001.

• **Step 2: Gateway & Authentication (FastAPI)**: main.py receives request. Calls require_student("S1001") which queries SQLite students table. Validates student existence. Generates 12-char hex trace_id (e.g. 4a8f91b2c3d4).

• **Step 3: Graph Invocation (LangGraph)**: main.py invokes graph.invoke({"question": "...", "student_id": "S1001", "as_of_date": "2026-10-09"}). The State dictionary enters the router node.

• **Step 4: Intent Classification (Router Node)**: router() in graph.py normalizes query text to lowercase. Extracts course code "CS201" via regex. Matches phrase "my attendance" and categorizes intent as personal_data.

• **Step 5: Retrieval Node (Vector & Precedence)**: retrieve_node() runs. For personal_data, vector search is skipped or scoped. Candidates pass to source_precedence.resolve().

• **Step 6: Answer Node & Tool Execution**: answer() evaluates intent personal_data. Calls attendance("S1001", "CS201") in tools.py. Tool executes SQLite query: SELECT * FROM attendance WHERE student_id="S1001" AND course_code="CS201". Fetches classes_held=28, classes_attended=22. Calculates percentage = round(22/28*100, 2) = 78.57%. Constructs tool output array.

• **Step 7: Answer Assembly & Explanation Layer**: answer() sets answer_type="calculated" and formats text: "Your attendance in CS201 is 78.57% (22 attended / 28 held)." Calls llm.explain(). In Mock mode, returns verified text instantly.

• **Step 8: Audit Logging & Storage (SQLite)**: main.py constructs audit dictionary containing trace_id, timestamp, student_id, question, intent, tools_invoked, answer_type, latency_ms. Executes INSERT INTO audits VALUES (?,?,?) in SQLite.

• **Step 9: UI Render (React)**: FastAPI returns AskResponse JSON. React updates state. Renders "calculated" green badge, trace ID, latency (e.g. 18 ms), response text, and interactive JSON box displaying tool outputs.

4. Comprehensive Interview Questions & Detailed Answers

Below are 25 in-depth technical, architectural, and behavioral interview questions tailored for this project:

Q1: What is the core problem this project solves, and why is a hybrid RAG + Deterministic approach better than a pure LLM chatbot?

In university domain applications, answers must be zero-hallucination and strictly policy-compliant. Pure LLMs struggle with precise math (calculating attendance percentages), multi-document supersession, and authorization boundaries. Our project uses LangGraph for workflow routing, ChromaDB for vector retrieval, pure Python for source precedence and math tools, and SQLite for atomic data store. The LLM (or Mock engine) is restricted to explaining verified evidence, ensuring 100% accuracy.

Q2: What is the role of LangGraph in this application compared to standard LangChain chains?

LangChain chains are linear sequences of calls. LangGraph provides a cyclic state machine (`StateGraph`). It allows conditional routing, state persistence, step-by-step node execution (Router -> Retrieve -> Answer), and explicit state validation between nodes. This ensures that intent classification, document retrieval, tool invocation, and evidence rendering happen in controlled, isolated steps.

Q3: How does the system handle security and prevent cross-student data leaks?

Authentication is enforced via the HTTP `X-Student-Id` header validated against SQLite. If Student S1001 asks about S1002's attendance, the Router node identifies the target ID discrepancy and immediately sets `answer_type='refused'`, returning an authorization error. Prompt injections attempting to trick the LLM into revealing another student's data are neutralized because database queries are parameterized by the validated header ID, not natural language text.

Q4: Explain how the Source Precedence Engine resolves conflicts between multiple policy documents.

The precedence engine (`app/policy/source_precedence.py`) executes a 5-step Python algorithm: 1) Filters documents valid on `as_of_date`. 2) Checks explicit supersession tags (if Doc B supersedes Doc A, A is removed). 3) Ranks by authority level (Level 1 Regulations beat Level 2 Circulars). 4) Selects the most recent `effective_from` date among ties. 5) If a tie remains, it flags `conflict_flagged` and cites both documents, avoiding guessing.

Q5: Why is attendance percentage dynamically calculated in Python instead of stored in the database?

Storing calculated percentages in SQLite causes data drift whenever total `classes_held` increases. To maintain database normalization and data integrity, we store raw integers (`classes_attended` and `classes_held`) with check constraints (`classes_attended <= classes_held`). The `attendance()` tool calculates `round((attended / held) * 100, 2)` at runtime.

Q6: What is section-aware chunking, and why was it necessary for documents like lab manuals?

Standard fixed-character chunking (e.g. 500 characters) cuts text indiscriminately, often splitting headings or mixing adjacent lab experiments. Our parser recognizes section patterns (e.g. `EXPERIMENT 1`, `SECTION 7`, `MODULE 3`) using regex, keeping heading units together. When an exact section is requested, the system returns that specific heading block rather than an arbitrary vector slice.

Q7: How does the hypothetical 'what-if' examination evaluation work without corrupting student records?

When a user asks 'If I pass my supplementary exam, will I be eligible?', the graph classifies the query as `multi_step`. It calls the `eligibility()` tool passing `hypothetical_pass=True`. The tool evaluates rule thresholds against an in-memory dictionary copy of the student's status without issuing an UPDATE query to SQLite. The database remains untouched.

Q8: How does document ingestion handle idempotent re-uploads?

When a document with an existing `doc_id` is re-uploaded via `POST /ingest`, the endpoint executes `DELETE FROM documents WHERE doc_id=?` and deletes all matching chunks from ChromaDB before indexing the new text. This prevents stale duplicate chunks from accumulating in the vector store.

Q9: What is the purpose of the `as_of_date` parameter in `POST /ask`?

Policy rules and circulars have temporal validity (`effective_from` and `effective_to`). Passing an `as_of_date` allows historical or future policy evaluation (e.g., 'What was the rule in 2024?' or 'What will be the rule next semester?'). The precedence engine filters documents based on whether `effective_from <= as_of_date < effective_to`.

Q10: What happens if Ollama / local LLM is offline or unavailable?

The project features a `MOCK_LLM=true` mode. In mock mode, the application uses an evidence-grounded fallback synthesizer and deterministic hashing embeddings. RAG retrieval, LangGraph routing, tool calculations, and audit logging function at 100% capacity with zero latency and zero reliance on external LLM APIs.

Q11: Explain the structure of an audit log entry in this system.

Each request generates a 12-char hex `trace_id`. The audit JSON stored in SQLite records: trace_id, ISO timestamp, student_id, original question, intent type, as_of_date, retrieved source IDs, selected source IDs, precedence decision reason, tools invoked with inputs/outputs, applied rule IDs, answer type, model used, latency in ms, and final answer. Prompts and CoT are excluded.

Q12: How does the system handle unanswerable queries like 'Scholarship amount for studying in Antarctica'?

The router identifies non-existent topics or ChromaDB retrieval fails to return chunks above the relevance threshold. The state machine sets `answer_type='not_found'` and returns 'I could not find this information in authoritative university sources.' This prevents hallucinated answers.

Q13: How is database consistency maintained in SQLite?

We enable `PRAGMA foreign_keys=ON` and apply CHECK constraints across tables. For example: `cgpa BETWEEN 0 AND 10`, `classes_held > 0`, `classes_attended <= classes_held`, and `total_marks = internal_marks + external_marks`. Invalid inserts or updates are rejected at the database engine level.

Q14: What vector embedding model is configured for ChromaDB?

The primary embedding model is `all-MiniLM-L6-v2` from Sentence Transformers (384-dimensional dense vectors). In lightweight demo/mock mode, a custom 128-dimensional SHA-256 term-frequency hashing vectorizer is used as a fallback.

Q15: How does `POST /reindex` work?

If the chunking algorithm, overlap size, or embedding model is updated, calling `POST /reindex` fetches all original raw texts from the SQLite `documents` table, clears the Chroma collection, re-chunks all texts with updated settings, and re-embeds them into ChromaDB without requiring manual file re-uploads.

Q16: How do you evaluate the quality of the system?

`evaluation/evaluate.py` runs 20 benchmark test cases covering policy facts, personal attendance, eligibility, authorization attempts, and unanswerable questions. It computes correctness %, citation accuracy %, tool result accuracy %, abstention accuracy %, retrieval hit rate @ k=3/5, and P50/P95 latency.

Q17: What are the main limitations of pypdf during ingestion?

`pypdf` extracts text streams from digital PDFs. However, if a PDF contains scanned images of text or complex multi-column tables, `pypdf` returns empty or garbled text. For production, an OCR engine like Tesseract or Amazon Textract would be integrated.

Q18: How does the frontend communicate with the backend?

The React frontend uses standard `fetch` HTTP requests to communicate with FastAPI. Authentication is passed via the custom header `X-Student-Id`. The response payload includes typed fields (`answer`, `citations`, `tool_results`, `trace_id`), enabling the UI to render source badges and tool execution boxes.

Q19: How would you scale this architecture to support 100,000 active students?

1) Replace SQLite with PostgreSQL (Amazon RDS) with read replicas. 2) Replace local ChromaDB with a distributed vector store like Qdrant or Milvus. 3) Deploy FastAPI on Kubernetes (EKS) behind an ALB. 4) Use Redis for session state and rate limiting. 5) Deploy Ollama or vLLM on dedicated GPU nodes.

Q20: Why did you choose FastAPI over Flask or Django?

FastAPI provides native asynchronous async/await support, automatic Pydantic data validation, high performance (built on Starlette and Uvicorn), and auto-generated OpenAPI/Swagger documentation (`/docs`), making it ideal for modern microservices.

Q21: How do you protect against SQL Injection?

All database queries in `tools.py`, `db.py`, and `main.py` use parameterized SQL queries with `?` placeholders (e.g. `c.execute('SELECT * FROM students WHERE student_id=?', (sid,))`). User input is never concatenated into raw SQL strings.

Q22: What happens if a student has failing marks in a course but attendance above 75%?

The `eligibility()` tool evaluates both rules from `rule_registry`. Attendance rule requires `>= 75%`, result rule requires `result == 'FAIL'`. If both are satisfied, `eligible` is evaluated to `True` for the supplementary examination.

Q23: How are duplicate uploads of the same text under different document IDs handled?

The `retrieve_node` in LangGraph computes a whitespace-normalized text fingerprint. Identical evidence uploaded under different IDs is recognized as a duplicate and collapsed to a single representative before reaching conflict resolution.

Q24: What metric demonstrates the performance of retrieval?

The evaluation framework tests retrieval top-k precision. In our dataset benchmarks, Top-K=3 achieves a retrieval hit rate over 90%, with median response latency under 50ms in Mock mode.

Q25: If asked: 'What was your biggest technical challenge in this project?', how should you answer?

Answer: 'The biggest challenge was handling policy document supersession and date applicability. Standard vector search returns chunks based purely on semantic similarity, which often fetched old policy circulars instead of newly issued regulations. I solved this by building a custom Python Source Precedence Engine that metadata-filters retrieved chunks by effective date, authority tier, and explicit supersession tags before passing evidence to the response layer.'

End of Technical Guide & Interview Preparation Document